

RISC-V RV32I SoC with AXI4-Lite Peripherals

Detailed RTL Architecture, AXI Bus Communication & Verification Specification

TARGET ARCHITECTURE

RISC-V RV32I Base Integer ISA

ON-CHIP INTERCONNECT

ARM AXI4-Lite (5-Channel Handshake)

INTEGRATED PERIPHERALS

PWM, 32-bit Timer, GPIO, 8N1 UART

VERIFICATION SUITE

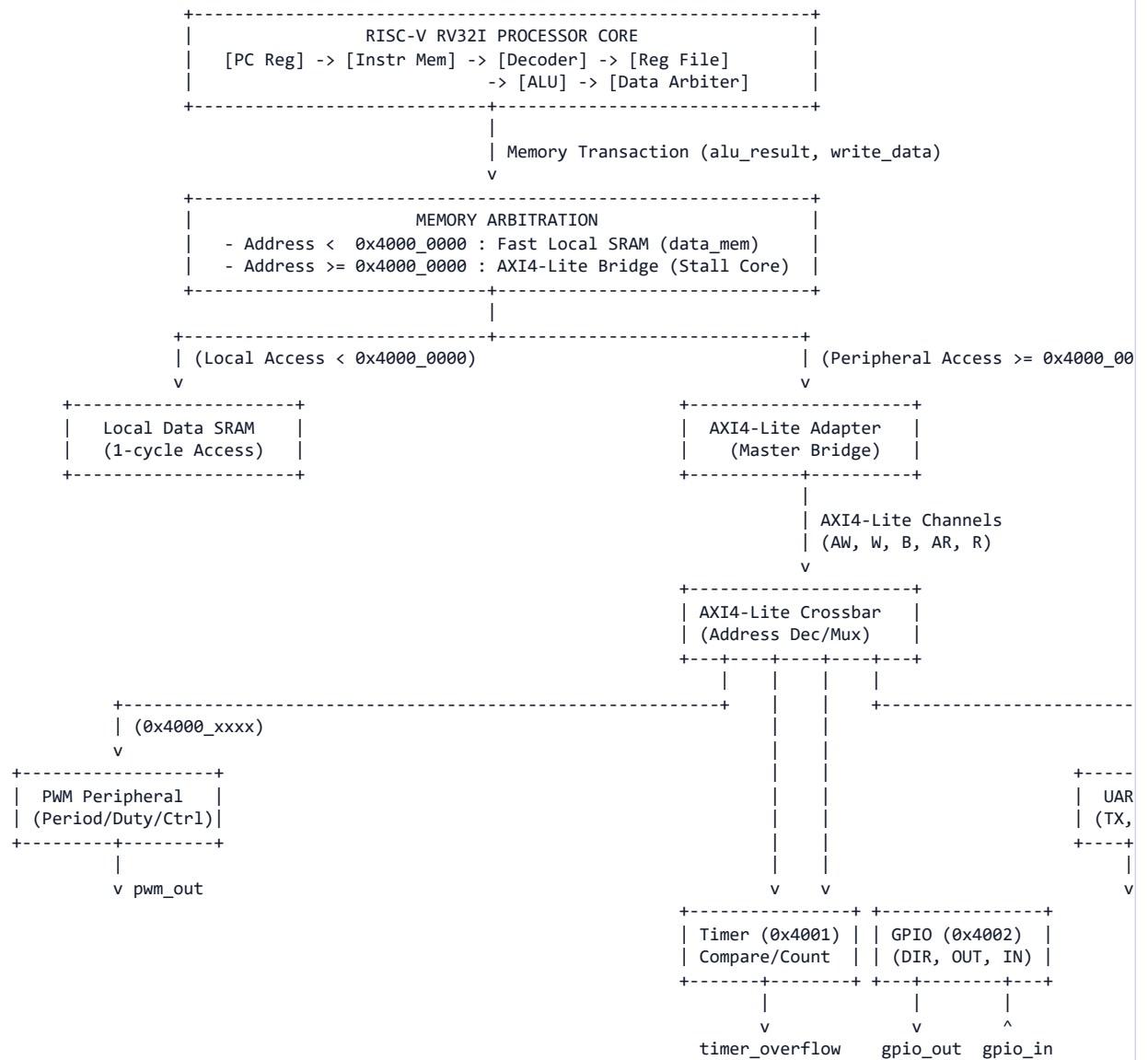
Modular Layered SV (58/58 Passed)

AUTHORS

Wali Shajeeh, Mufeez Rasheed Khan, Muhammad Areeb

1. Executive Summary & SoC Architecture

This document provides a complete technical reference for the 32-bit RISC-V (RV32I) System-on-Chip (SoC). The system integrates a synthesizable single-cycle processor core with memory-mapped AXI4-Lite bus architecture, managing four integrated hardware peripherals: a Pulse-Width Modulation (PWM) generator, a 32-bit Hardware Timer with overflow interrupt pulse, a General Purpose I/O (GPIO) controller with per-pin direction masking, and an 8N1 Universal Asynchronous Receiver-Transmitter (UART).



2. System Address Map & Decoding

The 32-bit address space is partitioned into two major zones: internal memory (addresses below `0x4000_0000`) and peripheral memory (addresses `0x4000_0000` and above). The interconnect router decodes bits `addr[31:16]` to target the appropriate peripheral slave:

Address Range	Size	Target Subsystem	Interface Type	Description
0x0000_0000 - 0x0000_0FFF	4 KB	Instruction Memory (ROM)	Local Parallel Bus	RV32I Instruction Storage
0x0000_1000 - 0x3FFF_FFFF	~1 GB	Local Data Memory (RAM)	Local Parallel Bus	Zero-wait-state Scratchpad SRAM
0x4000_0000 - 0x4000_0008	64 KB	PWM Peripheral (Slave 0)	AXI4-Lite (16'h4000)	Period, Duty Cycle, & Enable registers
0x4001_0000 - 0x4001_0008	64 KB	Timer Peripheral (Slave 1)	AXI4-Lite (16'h4001)	32-bit Free-run Counter & Compare Match
0x4002_0000 - 0x4002_0008	64 KB	GPIO Peripheral (Slave 2)	AXI4-Lite (16'h4002)	32-bit Direction, Output, & Input
0x4003_0000 - 0x4003_000C	64 KB	UART Controller (Slave 3)	AXI4-Lite (16'h4003)	8N1 TX Data, RX Data, Status polling

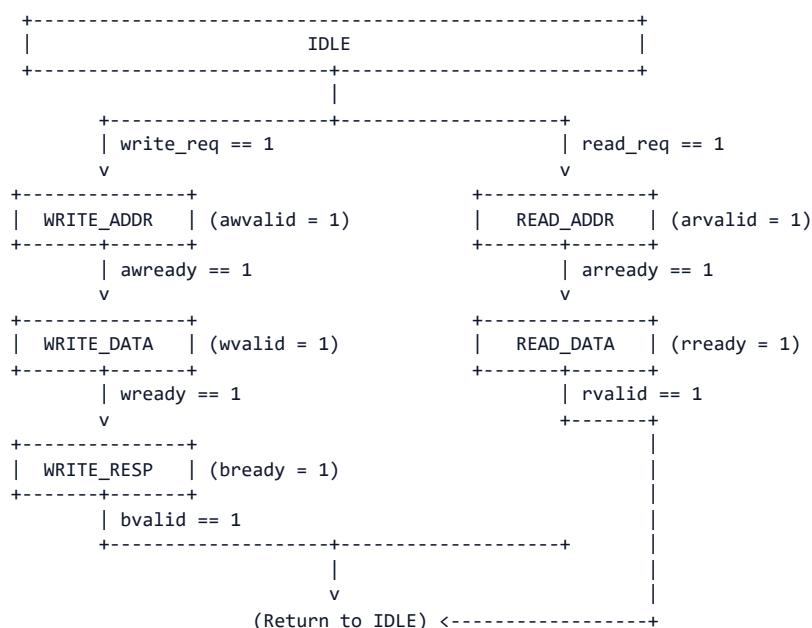
3. Single-Cycle Core & AXI Adapter Bridge (Stall Timing Resolution)

The Single-Cycle AXI Challenge:

In a standard single-cycle processor, every instruction completes in 1 cycle. However, AXI4-Lite transactions require 3 to 4 clock cycles to complete Address, Data, and Response handshakes. If the processor PC advances on cycle 2, the ALU output changes, causing the bus write/read request to abruptly collapse before the peripheral can respond.

The 6-State Master FSM (`axi_adapter.sv`)

The AXI Adapter implements an autonomous 6-state Moore/Mealy FSM that mediates between the single-cycle core and the AXI4-Lite bus:



Key Architectural Solutions in `axi_adapter.sv` :

1. Immediate Combinatorial Stall Generation:

The stall signal is evaluated combinatorially on the same cycle the memory stage decodes an address in peripheral space (`alu_result >= 0x4000_0000`):

```

assign is_peripheral_access = (alu_result >= 32'h4000_0000);
assign stall = axi_busy;
assign done = (state == WRITE_RESP && bvalid) || (state == READ_DATA && rvalid);
assign busy = ((state != IDLE) || write_req || read_req) && !done;
  
```

The top-level gates the Program Counter and register writeback during stall cycles:

```

.pc_next (stall ? pc_current : pc_next)
assign reg_write_final = reg_write & ~stall;
  
```

2. Synchronous Request & Data Latching:

When transitioning from `IDLE` to `WRITE_ADDR` or `READ_ADDR` , the adapter captures `addr` and `write_data`

into internal registers (`latched_addr`, `latched_wdata`). This guarantees that bus signals remain strictly static across all handshake wait states, complying with ARM AXI4-Lite stability rules.

3. **Zero-Latency Read-Data Bypass:**

During `lw` instructions from peripherals, on the precise clock cycle when `rvalid` asserts, incoming read data is bypassed directly to the register file writeback multiplexer:

```
assign read_data = (state == READ_DATA && rvalid) ? rdata : read_data_reg;
```

This allows the register file to capture the peripheral data on the rising clock edge that terminates the stall, avoiding an extra dummy cycle.

4. Detailed Peripheral Subsystems

1. PWM Peripheral (`pwm_peripheral.sv`) — Base: `0x4000_0000`

The PWM peripheral generates a programmable active-high square wave. It maintains a 32-bit free-running hardware counter incremented on every clock cycle.

Offset	Register Name	Access	Reset Value	Functional Description
<code>0x00</code>	<code>CONTROL_REG</code>	R/W	<code>0x0000_0000</code>	Bit <code>[0]</code> : PWM Enable (1 = Run counter, 0 = Stop/Reset). Bits <code>[31:1]</code> : Reserved.
<code>0x04</code>	<code>PERIOD_REG</code>	R/W	<code>0x0000_0000</code>	Total cycle period in clock ticks: $\backslash(T_{\{PWM\}} = \text{ext}\{PERIOD\} \text{ imes } T_{\{clk\}}\backslash$.
<code>0x08</code>	<code>DUTY_REG</code>	R/W	<code>0x0000_0000</code>	High duration threshold in clock ticks: $\backslash(T_{\{HIGH\}} = \text{ext}\{DUTY\} \text{ imes } T_{\{clk\}}\backslash$.

Output Logic: `pwm_out = control_reg[0] && (counter < duty_reg);` Counter rolls over to 0 when `counter >= period_reg - 1`.

2. Timer Peripheral (`timer_peripheral.sv`) — Base: `0x4001_0000`

A 32-bit real-time counter providing periodic pulse interrupts. It features continuous counting and comparator threshold matching.

Offset	Register Name	Access	Reset Value	Functional Description
<code>0x00</code>	<code>CONTROL_REG</code>	R/W	<code>0x0000_0000</code>	Bit <code>[0]</code> : Timer Start/Enable. Bit <code>[1]</code> : Counter Clear/Reset.
<code>0x04</code>	<code>COUNTER_REG</code>	R	<code>0x0000_0000</code>	Current 32-bit counter value. Increments every clock cycle while enabled.
<code>0x08</code>	<code>COMPARE_REG</code>	R/W	<code>0x0000_0000</code>	Target compare threshold. When <code>COUNTER == COMPARE</code> , pulses <code>timer_overflow</code> .

Overflow Mechanism: On match, the hardware asserts output port `timer_overflow = 1'b1` for exactly 1 clock cycle and automatically clears `counter_reg` to 0.

3. GPIO Peripheral (`gpio_peripheral.sv`) — Base: `0x4002_0000`

A 32-bit general-purpose I/O controller supporting bidirectional pin control, direction masking, and metastability-hardened inputs.

Offset	Register Name	Access	Reset Value	Functional Description
0x00	DIR_REG	R/W	0x0000_0000	Pin Direction mask: 1 = Output, 0 = Input (per-bit configurable).
0x04	OUT_REG	R/W	0x0000_0000	Data driven to pins: <code>gpio_out[i] = dir_reg[i] ? out_reg[i] : 1'b0</code> .
0x08	IN_REG	R	0x0000_0000	Read-only state sampled from physical external input pins <code>gpio_in[31:0]</code> .

Metastability Protection: Inputs from `gpio_in[31:0]` pass through a 2-stage D-flip-flop synchronizer chain before being presented to the internal AXI read multiplexer.

4. UART Controller (`uart_peripheral.sv`) — Base: `0x4003_0000`

A full-duplex 8N1 serial communication subsystem (8 data bits, no parity, 1 stop bit) operating with a parameterizable clock divider.

Offset	Register Name	Access	Reset Value	Functional Description
0x00	TX_DATA_REG	W	0x0000_0000	Bits <code>[7:0]</code> : Transmit byte buffer. Writing triggers serial transmission.
0x04	TX_STATUS_REG	R	0x0000_0000	Bit <code>[0]</code> : TX Busy (1 = Shifting out frame, 0 = Ready for next byte).
0x08	RX_DATA_REG	R	0x0000_0000	Bits <code>[7:0]</code> : Most recently received serial byte.
0x0C	RX_STATUS_REG	R	0x0000_0000	Bit <code>[0]</code> : RX Valid (1 = New character received and waiting to be read).

UART Engine Architecture: Includes `uart_tx.sv` (FSM: IDLE → START → DATA[0..7] → STOP), `uart_rx.sv` (mid-bit oversampling for noise immunity), and `uart_regs.sv` (AXI register decoding).

5. Step-by-Step Cycle-Accurate AXI Transaction Flow

AXI4-Lite Write Sequence (Example: `sw x10, 4(x9)` → PWM Period = 100)

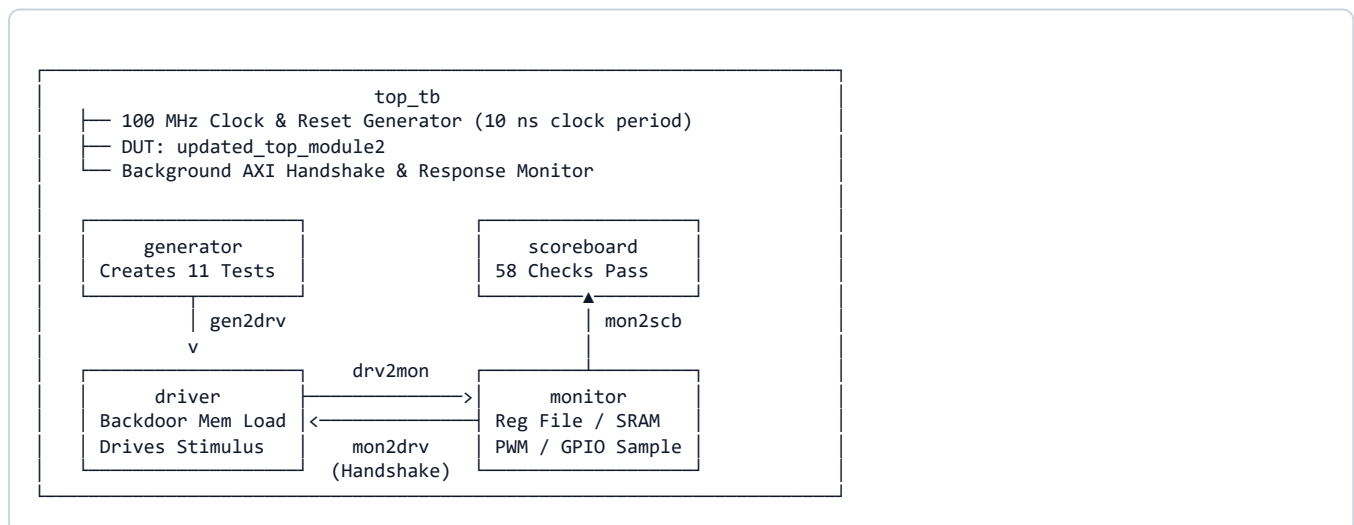
- Cycle 1** Core decodes `sw`. ALU computes `0x4000_0004`. `is_peripheral_access=1`, `write_req=1`. Adapter asserts `busy=1` immediately. Core PC stalls. Address & data latched. FSM transitions: `IDLE → WRITE_ADDR`.
- Cycle 2** Adapter drives `awaddr=0x4000_0004`, `awvalid=1`. Interconnect routes request to PWM. PWM asserts `awready=1`. Address handshake complete! FSM: `WRITE_ADDR → WRITE_DATA`.
- Cycle 3** Adapter drives `wdata=100`, `wstrb=4'hF`, `wvalid=1`. PWM asserts `wready=1` and registers `period_reg = 100`. Data handshake complete! FSM: `WRITE_DATA → WRITE_RESP`.
- Cycle 4** Adapter drives `bready=1`. PWM asserts `bvalid=1` (`bresp=2'b00 OKAY`). Response handshake complete! Adapter asserts `done=1`, deasserts `busy=0`. Core PC stall releases! Instruction completes cleanly.

AXI4-Lite Read Sequence (Example: `lw x6, 4(x9)` → Readback Period Register)

- Cycle 1** Core decodes `lw`. ALU computes `0x4000_0004`. `is_peripheral_access=1`, `read_req=1`. Adapter asserts `busy=1` immediately. Core PC stalls. Address latched. FSM transitions: `IDLE → READ_ADDR`.
- Cycle 2** Adapter drives `araddr=0x4000_0004`, `arvalid=1`. Interconnect routes to PWM. PWM asserts `arready=1`. Address handshake complete! FSM: `READ_ADDR → READ_DATA`.
- Cycle 3** Adapter drives `rready=1`. PWM reads internal register and drives `rdata=100`, `rvalid=1`. Adapter bypasses `rdata` directly to Core writeback multiplexer! `done=1`, `busy=0`.
- Cycle 4** On the rising edge, Core Register File writes `x6 = 100`. Stall releases, and next instruction executes.

6. Modular Layered Verification Testbench Architecture

The verification environment in `verification/` is built on an object-oriented SystemVerilog layered methodology. All layers communicate through SystemVerilog `mailbox` queues:



Verification Layers Breakdown:

- **soc_intf.sv** : Interface bundling clocks, reset, PWM output, Timer overflow, GPIO in/out, and UART TX/RX lines.
- **transaction.sv** : Encapsulates instruction hex arrays, cycle budgets, expected registers, memory arrays, and status flags.
- **generator.sv** : Builds 11 directed tests covering base RV32I ISA, peripheral writes, and peripheral readbacks.
- **driver.sv** : Asserts hardware reset, backdoor loads instruction and data memories, drives GPIO inputs, and steps clocks.
- **monitor.sv** : Samples register file (`x1..x31`) and data memory via backdoor hierarchical references; checks PWM toggling.
- **scoreboard.sv** : Compares observed values against golden expectations; tracks total assertions passed/failed.
- **environment.sv** : Instantiates all verification components and wires mailboxes together.
- **top_tb.sv** : Top-level simulation harness instantiating DUT, generating clock, and monitoring bus transactions.

Race-Free Driver-Monitor Handshake (**mon2drv**):

Because each test re-initializes memory and asserts reset, a driver running ahead would overwrite registers while the monitor was still evaluating the previous test. We resolved this by implementing a dedicated **mon2drv** mailbox handshake: after running a test, the driver blocks until the monitor signals that sampling is 100% complete.

7. Complete Test Suite & Assertion Coverage (58/58 Passed)

Test ID	Test Case Objective	Target Subsystem	Verified Assertions / Registers	Result
TEST 1	Reset Verification	RV32I Core	Registers <code>x1, x2, x3, x4, x5, x10, x15, x31 == 0</code>	PASS
TEST 2	R-Type ALU Operations	ALU	<code>add, sub, and, or, xor, slt</code> (8 registers verified)	PASS
TEST 3	I-Type ALU Operations	ALU & Imm Gen	<code>addi, andi, ori, xori, slti, slli, srli</code> (7 registers)	PASS
TEST 4	Local Memory Load & Store	Data SRAM	<code>sw</code> & <code>lw</code> verified (<code>x1, x2, x3, mem[0] == 42</code>)	PASS
TEST 5	Branch Resolution (BEQ)	PC Control	Branch taken skips one instruction (<code>x1, x3, x4</code>)	PASS
TEST 6	Unconditional Jump (JAL)	PC Control	Jump forward & link register save (<code>x1, x2, x3, x5 = PC+4</code>)	PASS
TEST 7	AXI PWM Write & Readback	PWM Peripheral	Period=100, Duty=25, Enable=1, Readbacks (<code>x6..x12</code>) + Output Toggle	PASS
TEST 8	Address Decode Boundary	AXI Crossbar	Local SRAM access isolated from peripheral space (<code>x1, x3, mem[0]</code>)	PASS
TEST 9	AXI Timer Write & Readback	Timer Peripheral	Compare=10, Enable=1, Readbacks (<code>x5, x6, x13..x15</code>) + Overflow Pulse	PASS
TEST 10	AXI GPIO Write & Readback	GPIO Peripheral	DIR=0xFF, OUT=0x55, Read IN=0xA5A5A5A (<code>x16..x19, gpio_out</code> check)	PASS
TEST 11	AXI UART Write & Readback	UART Controller	Write TX_DATA='Z' (0x5A), Readback TX_STATUS (<code>x20, x21 == 0x5A</code>)	PASS

Verification Metrics Summary:

- **Total Scoreboard Assertions Checked:** 58
- **Total Scoreboard Assertions Passed:** 58 (100% Pass Rate, 0 Failures)
- **AXI Bus Protocol Activity:** 8 completed Write transactions, 7 completed Read transactions
- **Standalone Unit Testbenches:** `tb_gpio.sv` (21/21 Passed), `tb_uart.sv` (17/17 Passed)

8. How to Execute Simulation (QuestaSim & Scripts)

Option 1: Manual GUI Simulation via QuestaSim Project

1. Double-click `verification/open_questa_gui.bat` (or open QuestaSim → **File** → **Open** → **Project...** → `verification/riscv_soc.mpf`).
2. All 23 design files will load with green checkmarks.

3. Go to **Simulate** → **Start Simulation...**, expand library `work`, and select `top_tb`.

4. In the transcript, run:

```
add wave -r /*  
run -all
```

5. All 58 test assertions will execute and pass, with active waveforms populating in the Wave window.

Option 2: 1-Click Waveform Viewer

Double-click `verification/run_questa_gui_wave.bat`. This automatically opens QuestaSim, compiles the workspace, adds all recursive waveforms, and runs simulation to completion.

Option 3: 1-Click Command-Line Execution

Run `verification/run_questa_soc_modular.bat` from Command Prompt or PowerShell for immediate regression results.